AI Network Intrusion Detection System

Project handoff guide for teammates and AI assistants

Version: November 8, 2026
Project type: Three-week, five-person undergraduate AI class project
Repository: `ai-network-intrusion-detection` (team's GitHub repository; exact URL is not included)
Read this document first when helping someone who is stuck. It explains the goal, ownership rules, setup, input/output contracts, and what to check before changing code.

> **The project in one sentence:** Compare four Python machine-learning approaches for deciding whether a record in the prepared UNSW-NB15 network-traffic dataset is **normal (0)** or **attack/suspicious (1)**, and show their results in a small Streamlit dashboard.

What the project is - and is not

The team is building an **offline class demonstration**, not a live network monitor or a production intrusion prevention system. The objective is to understand the dataset, train and evaluate different AI methods, compare trade-offs such as missed attacks versus false alarms, and explain the results. Success means reproducible scripts, valid outputs, a working comparison dashboard, and honest documentation of limitations; it does **not** require near-perfect accuracy or a deployed cybersecurity product.

Important current-status note

As of this handoff, the user confirmed that **Python 3.11.9 is available** and that `.venv` was created on Windows. PowerShell initially blocked activation; the session-only execution-policy workaround was provided. **Do not assume** that package installation, dataset download, starter-kit merge, model training, or GitHub commits have already succeeded. Verify the local state before deciding what to do next.

First things an AI assistant should do

1. **Identify the teammate's role** (Person 1-5), the specific problem, and the last successful command. Do not demand a full project explanation again.
2. **Check the actual repository/files if available**; this handoff describes the starter kit, but a teammate's branch may have changed.
3. Give **small, copy-pasteable Windows PowerShell steps**, explain what each step accomplishes, and show how to verify it worked.
4. Respect folder ownership and the common data/output contract. Avoid edits in another teammate's code or shared utilities unless the group agrees.
5. If there is an error, ask for the **exact error text, command, and relevant file** rather than guessing. Never invent test scores or claim a command was executed without evidence.

1. Five people, five independent workstreams

Each teammate can start immediately on their own machine. **No one waits for Person 1 to preprocess data or Person 5 to finish the dashboard**: the starter kit provides stable shared helpers, and Person 5 has tiny fake demonstration results.

Owner	Folder they change	Role / deliverables	Main real-data command
Person 1	`person1_baseline/`	Explore dataset, create 1-2 charts, train Logistic Regression; write `results/p1.csv` and explanation	`python person1_baseline/analyze.py` then `python person1_baseline/train.py`
Person 2	`person2_random_forest/`	Train Random Forest, try one parameter change, explain results; write `results/p2.csv`	`python person2_random_forest/train.py`
Person 3	`person3_svm/`	Train linear SVM (`LinearSVC`), examine speed / convergence; write `results/p3.csv`	`python person3_svm/train.py`
Person 4	`person4_isolation_forest/`	Train Isolation Forest on *normal training rows only*; discuss anomalies; write `results/p4.csv`	`python person4_isolation_forest/train.py`
Person 5	`person5_dashboard/`	Show model metrics and comparisons in Streamlit; capture a screenshot and write findings	`streamlit run person5_dashboard/app.py`

Every person has a folder-specific `README.md`. Read yours first and update its TODO notes with actual decisions, training sizes, metrics, and findings. The main `README.md` and `docs/TEAM_CHECKLIST.md` provide the overall checklist.

The three-week plan

- **Week 1 - Setup and first run:** Clone repo, create Python environment, test the starter script with synthetic practice data, download real CSVs, and understand your own files. Person 5 opens the demo dashboard immediately.
- **Week 2 - Working deliverables:** People 1-4 run their models on the real dataset, produce prediction CSVs, and make one understandable experiment or improvement. Person 5 adds or improves useful comparison views.
- **Week 3 - Finalize and integrate:** Each person checks their results, writes a brief explanation, and opens their own GitHub pull request. The dashboard loads available real prediction files; the team validates outputs, prepares a screenshot, and explains conclusions and limitations.

If someone is behind: They can still begin from their own README and send progress in the group chat. Other teammates continue. Missing results can be added to the dashboard later; do not substitute demo predictions for unfinished real results.

What counts as done for each person?

- **People 1-4:** Their `train.py` runs from the repository root; `results/pN.csv` is valid; their own `metrics.json` records real-data metrics; README lists training size, method/experiment, and a short interpretation. Person 1 also includes dataset charts/summary.
- **Person 5:** The Streamlit app opens, handles missing model CSVs, compares the available valid predictions, distinguishes fake demo data from real data, and has a screenshot and 3-5 sentence comparison.
- **Everyone:** Their own work is committed to a separate branch and submitted in a pull request without accidentally adding the dataset, virtual environment, or files owned by another person.

2. Dataset and results: the agreement nobody should break

Source files and labels

Use the **prepared UNSW-NB15 split** (not the large raw capture files):

```
data/raw/UNSW_NB15_training-set.csv   # 175,341 data records
data/raw/UNSW_NB15_testing-set.csv    # 82,332 data records
```

The files are downloaded locally and **not committed** to GitHub. The label field means `0 = normal` and `1 = attack` in the original dataset. The original UNSW research page is `https://research.unsw.edu.au/projects/unsw-nb15-dataset` and a prepared-file mirror is listed in `data/README.md`.

Use training CSV to fit; use testing CSV only for final predictions/evaluation. The trained preprocessing steps must also be fitted on training data only. The supplied helper pipelines do this. Models must never receive **`id`, `label`, or `attack_cat` as input features**, because those fields could leak the answer or metadata rather than teach intrusion patterns.

Shared prediction file format

Each model saves one CSV with **exactly three columns**, in this order:

```
row_id,y_true,y_pred
0,0,0
1,1,1
2,1,0
```

The rows shown are **format examples**, not real predictions. For a real run:

- `row_id` = original zero-based **test row position**, from `0` through `82331`, with no shuffling.
- `y_true` = original test label (0 or 1); `y_pred` = model's answer (0 or 1).
- The file contains **82,332 data rows** plus a header, and all model results refer to the **same** test observations.
- Model output paths are `results/p1.csv`, `results/p2.csv`, `results/p3.csv`, and `results/p4.csv`. Do not rename them or change column names.
- Run `python scripts/validate_results.py` to check existing outputs. The script checks schema, row IDs, allowed labels, and alignment across present files. Also verify that real runs actually contain **82,332** data rows; the validator does not by itself enforce the official expected count.

Why comparability matters

Person 5 calculates **accuracy, precision, recall, and F1-score** from each model's `y_true` and `y_pred`. In security, *recall* reflects how many labeled attacks were detected, while *precision* reflects how often a predicted attack was actually labeled an attack. A false positive is normal traffic flagged as an attack; a false negative is an attack missed by the model. Report trade-offs rather than chasing a single score.

Important differences among models

- **Logistic Regression / Random Forest / linear SVM** learn from labeled normal and attack training rows (supervised learning).
- **Isolation Forest** learns from normal training rows only (anomaly detection). Its raw `+1` means typical and `-1` means anomalous. The starter converts these to `0 = normal` and `1 = suspicious`. **An anomaly is not guaranteed to be a real attack.**
- The starters default to **up to 30,000 training records** for easier initial runs; the real test prediction still covers all 82,332 records. Add `--max-train-rows 0` to a model command only if the computer can handle training on all rows. Report the actual training count used.

3. Repository map and where to look first

```
ai-network-intrusion-detection/
  README.md                    # project overview
  QUICKSTART_WINDOWS.md       # first-time Windows setup
  CONTRIBUTING.md             # Git workflow and folder rules
  requirements.txt            # pinned-range Python libraries
  data/README.md              # dataset links and naming
  data/raw/                    # real CSVs, ignored by Git
  data/sample/                 # generated practice CSVs, ignored
  project_utils/core.py       # stable shared loading/preprocessing/output
  scripts/check_setup.py      # verify Python packages and files
  scripts/download_dataset.py # optional real data downloader
  scripts/smoke_test.py       # runs all 4 models on practice data
  scripts/validate_results.py # validates available predictions
  person1_baseline/           # Person 1 code and README
  person2_random_forest/      # Person 2 code and README
  person3_svm/                # Person 3 code and README
  person4_isolation_forest/   # Person 4 code and README
  person5_dashboard/          # Person 5 app, README, demo CSVs
  results/                    # real p1.csv ... p4.csv, commit these
  scratch/                    # temporary practice outputs, ignored
  docs/TEAM_CHECKLIST.md      # catch-up checklist
  docs/TROUBLESHOOTING.md     # frequent errors
```

Starter-code design: Each model `train.py` imports the same functions in `project_utils/core.py`: `load_data`, `preprocess_for`, and `save_results` (and training-row selection where applicable). The helper is already present so all five roles can develop independently. The team does **not** need to implement a separate shared preprocessing task before beginning.

Default rule: Change your own person's files and your own `results/pN.csv`. The shared `project_utils/core.py`, top-level README, and `requirements.txt` should remain stable unless the team agrees on a separate coordinated change.

Beginner-friendly orientation for AI assistants

When a teammate says "I don't understand my part," do not explain all five machine-learning methods. Start with their role's README, point out the 1-2 scripts they own, walk through one starter command, explain what

output it creates, and then suggest one small improvement they can document. Distinguish **what the boilerplate already implements** from **what the student is expected to explore, verify, and write up**.

When a teammate says "My model is wrong," check these in order: Does the script run? Is it using the right dataset? Is there leakage? Does `results/pN.csv` preserve order? Do metrics make sense? Is there class imbalance, a convergence warning, or a modeling trade-off? Do not change the entire architecture immediately.

4. Windows / VS Code setup and first successful run

Run all commands from **the repository root** (the folder containing `requirements.txt`). Each teammate creates a separate local `.venv`; virtual environments are not shared through GitHub.

PowerShell: one-time local setup

```
python --version                 # expected: Python 3.11.x or compatible
python -m venv .venv
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python scripts/check_setup.py
```

The `Set-ExecutionPolicy` command is only needed if PowerShell blocks activation; `-Scope Process` applies to the **current PowerShell process**, not permanently. If the system prohibits that policy, run `.\.venv\Scripts\python.exe` directly rather than activating. `py` may not exist on everyone's computer: `python -m venv .venv` works if `python --version` succeeds.

Test everything without downloading real data

```
python scripts/smoke_test.py
```

This generates **synthetic** CSVs, runs starter versions of Persons 1-4, and checks the practice outputs. It is a setup test only; its scores **must not** appear in the final scientific comparison. Practice predictions go under `scratch/sample_results/` and are ignored by Git.

Obtain the real prepared dataset

```
python scripts/download_dataset.py
python scripts/check_setup.py
```

If the downloader fails, follow `data/README.md`, download the prepared training/testing CSVs manually, and place them in `data/raw/` with the exact filenames. Each teammate downloads their own local copy; **do not push raw datasets to GitHub**.

Role commands, after real data is available

```
# Person 1: dataset charts, then Logistic Regression
python person1_baseline/analyze.py
python person1_baseline/train.py
```

```
# Person 2: Random Forest
python person2_random_forest/train.py

# Person 3: linear SVM
python person3_svm/train.py

# Person 4: Isolation Forest
python person4_isolation_forest/train.py

# Person 5: dashboard, works even before model results exist
streamlit run person5_dashboard/app.py

# After model results have arrived
python scripts/validate_results.py
```

Only run the command(s) for your assigned role. Person 5 can select **Example data (practice)** first, then switch to **Actual model results** once real CSVs have been merged into their checkout. Real model runs also create `personN_folder/metrics.json`; the dashboard uses the prediction CSVs, not these JSON files.

What to share with an assistant when setup fails

Give the exact command, complete error message, terminal working directory (`pwd`), `python --version`, and whether `(.venv)` appears in the prompt. If a script says a file is missing, show the relevant `dir` output. Never paste passwords or GitHub access tokens.

---

5. Working asynchronously, GitHub, and troubleshooting

One person = one Git branch and one owned folder

After the starter kit is merged into `main`, each teammate starts from updated `main` and makes their own branch. Example for Person 2:

```
git switch main
git pull origin main
git switch -c person2-random-forest
# Make and test changes inside person2_random_forest/
git status
git add person2_random_forest/
git add results/p2.csv          # only after real results exist
git commit -m "Train Random Forest and document results"
git push -u origin person2-random-forest
```

Then open a pull request into `main`. **Never create that branch again with `-c` if it already exists**; switch to it instead. Never `git add .` blindly without checking `git status`. If a model result is not ready, commit the code first and add the CSV later.

Things not to commit

- `.venv/`, `data/raw/`, `data/sample/`, `scratch/`, model binaries or credentials.
- Fake demo/synthetic predictions as if they were final results.
- Other teammates' unfinished files, or modifications to shared helpers without coordination.

Frequent problems and first checks

Symptom	Most useful first check / fix
`py` not recognized	Use `python --version`, then `python -m venv .venv`.
Activation blocked	`Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`, then activate again.
`ModuleNotFoundError`	Activate `.venv`; run `python -m pip install -r requirements.txt`.
Missing dataset / wrong filename	Use `python scripts/download_dataset.py`; inspect `data/raw/` and `data/README.md`.
Import or path issue	Run from the **repo root** and check `pwd` / `dir`.
SVM slow or warns on convergence	Start with 30,000 or 5,000 training rows; inspect `max_iter` and record warnings.
Dashboard shows fake/old results	Select **Actual model results**; fetch merged teammates' CSVs via Git.
Dashboard can't compare CSVs	Run `python scripts/validate_results.py`; check column order, `row_id`, and `y_true`.
Git merge conflicts	Stop and inspect `git status`; avoid overwriting other people's work.

How an AI assistant should suggest changes

Prefer minimal changes that preserve this workflow. Provide the exact filename to edit and a brief explanation. State what command tests the change. Do not propose live packet capture, cloud accounts, Docker, a whole new UI, or a different dataset unless the teammate explicitly asks. Don't report a model as "accurate" or "finished" until real metrics are shown. If metrics are disappointing, help interpret or improve them transparently rather than tampering with the test set.

Useful group update format: `Person | Done | Next | Blocked | Branch or PR link`. This is enough for teammates to coordinate without meetings.

6. Copy-and-paste prompt for any AI assistant

Paste the text below into ChatGPT, Copilot Chat, or another assistant **and attach this PDF**. Fill in only the parts that apply.

> I am working on a five-person undergraduate AI project called **AI Network Intrusion Detection System**. Please read the attached **AI Project Handoff Guide** before helping me. We compare Logistic Regression, Random Forest, linear SVM, and Isolation Forest using the prepared UNSW-NB15 training/testing CSVs. Each person owns a separate folder; the group uses a fixed prediction CSV format and will combine results in a Streamlit dashboard.
>
> **My role:** Person [1/2/3/4/5].
> **My current task:** [What I am trying to finish].
> **My setup:** Windows PowerShell / VS Code, Python [version], virtual environment [active or not].
> **What I ran:** `[exact command]`.
> **What happened:** `[paste full error or output]`.
> **Relevant files:** `[paste the script or attach the files, if available]`.
>
> Please explain the issue at a level suitable for a CS student who is still learning. First identify what is already implemented in the starter kit, then walk me through **small changes in my own folder**. Give exact commands, explain why they work, and show how to verify my fix. Preserve the agreed dataset split, avoid label leakage, and keep `row_id,y_true,y_pred` unchanged. Do not invent code results; ask for the specific file or error text if you need more information.

Small issue-report checklist

When asking for help, include these five things if possible:

1. **Person / file:** e.g. "Person 3, `person3_svm/train.py`".
2. **Goal right now:** e.g. "Run the SVM on the prepared real dataset".
3. **Command + full output:** the exact PowerShell command and error text.
4. **What already works:** e.g. "`.venv` active; sample smoke test passed".
5. **What you want from AI:** e.g. "Explain the error, give the smallest fix, and show a test".

Final project quality checklist

- [ ] The team can explain the question being answered and how each ML method differs.
- [ ] All reported metrics come from actual prepared UNSW-NB15 test data, **not** synthetic demo rows.
- [ ] No training data leaks into test evaluation, and no target/leakage columns become model inputs.
- [ ] Each available real prediction CSV has the exact schema, correct row order, and 82,332 data rows.
- [ ] All team files have short README explanations, reproducible run commands, and documented limitations.
- [ ] The dashboard distinguishes fake and real predictions, works with missing models, and shows meaningful comparisons.
- [ ] Git branches and PRs are clean; no large raw datasets or local environments were committed.

Where to find more details: `README.md` -> your `personN_*/README.md` -> `docs/TEAM_CHECKLIST.md` -> `docs/TROUBLESHOOTING.md` -> `CONTRIBUTING.md`. The previously prepared `Network_Intrusion_Project_3_Week_Async_Plan.md` and PDF remain the planning reference. Treat the **actual files in the current Git branch** as the source of truth when they differ from this dated guide.